

Sicherheitsbericht Adressverwaltung

Stand: 4. Oktober 2026 · Version 2.3.0 · Frontend (Next.js), Backend (ASP.NET Core OData API), nginx, Docker Compose, Hilfsskripte

1. Zusammenfassung

Die Prüfung ergab 18 Befunde: 2 kritische, 6 hohe, 5 mittlere und 5 niedrige. 17 sind im Code behoben und am laufenden Stack überprüft. Einer ist nur im Code behoben: Das bisherige Standardpasswort des Admin-Benutzers gilt in der bestehenden Datenbank weiterhin (S-05); das ist bewusst akzeptiert.

Die zwei kritischen Befunde erlaubten den Zugriff auf alle Daten ohne Anmeldung: Jeder konnte sich über `/auth/register` selbst ein Konto anlegen, und die eingesetzte Next.js-Version liess sich am Seitenschutz vorbei aufrufen.

Die Adressverwaltung ist eine Demo-Anwendung ohne schützenswerte Daten. Deshalb sind zwei Punkte bewusst akzeptiert (Entscheid vom 4. Oktober 2026): die Geheimnisse in der Git-History auf GitHub und das alte Admin-Passwort in der bestehenden Datenbank. Fünf weitere Punkte bleiben als Restrisiken offen, weil sie sich nicht durch eine kleine Codeänderung lösen lassen (Abschnitt 4).

2. Umfang und Vorgehen

- Manuelle Durchsicht des Quellcodes: Controller, DbContext, Seeder, DTOs, Program.cs, lib/auth.ts, lib/api.ts, middleware.ts, Login-Formular, nginx.conf, docker-compose.yml, Dockerfiles, Skripte in scripts/ und migration/.
- Abhängigkeiten: `dotnet list package --vulnerable --include-transitive` und `npm audit`.
- Überprüfung der Korrekturen: 51 Backend-Integrationstests (davon 5 neu für diese Korrekturen) und Aufrufe gegen den neu gebauten Docker-Stack unter `https://localhost`.

Nicht Teil der Prüfung: automatisierte Penetrationstests mit externen Werkzeugen, die Inhalte der Git-History im Detail und die Infrastruktur ausserhalb des Docker-Stacks.

3. Befunde und Korrekturen

Nr.	Stufe	Befund	Korrektur	Status
S-01	Kritisch	Offene Registrierung: POST <code>/auth/register</code> war ohne Anmeldung erreichbar. Ein neu angelegtes Konto erhält ein gültiges Token und damit vollen Lese- und Schreibzugriff auf alle Adressen, Städte und Einstellungen.	[Authorize] auf der Registrierung: Nur angemeldete Benutzer können weitere Benutzer anlegen. Die Tests legen ihren Benutzer direkt in der Datenbank an.	Behoben
S-02	Kritisch	Next.js 14.2.5 mit bekannten Lücken, darunter die Umgehung der Middleware (GHSA-f82v-jwr5-mffw), die hier den gesamten Seitenschutz bildet, und Codeausführung über die Bildoptimierung (GHSA-2xp9-vwfh-vxw4). Die 14er-Linie erhält dafür keine Korrekturen mehr.	Upgrade auf Next.js 15.5.27 und React 19. <code>npm audit --omit=dev</code> meldet keine Lücken mehr. PostCSS per overrides auf eine korrigierte Version angehoben.	Behoben
S-03	Hoch	Geheimnisse im Repository: JWT-Schlüssel, NextAuth-Secret und Datenbankpasswort standen im Klartext in <code>docker-compose.yml</code> . Mit dem JWT-	Geheimnisse in die Datei <code>.env</code> ausgelagert (git-ignoriert, Vorlage <code>.env.example</code>); <code>docker -</code>	Behoben (History: O-01,

		Schlüssel lassen sich gültige Tokens für beliebige Benutzer fälschen.	compose.yml setzt sie als Variablen ein. Alle drei Werte neu erzeugt; das Datenbankpasswort wurde in der laufenden Datenbank geändert.	akzeptiert)
S-04	Hoch	Platzhalter-JWT-Schlüssel in appsettings.json, der ohne eigene Konfiguration stillschweigend verwendet wurde; dazu das Datenbankpasswort in derselben Datei.	Schlüssel und Passwort aus appsettings.json entfernt. Das Backend startet nicht, wenn Jwt:Key fehlt, kürzer als 32 Zeichen oder der alte Platzhalter ist.	Behoben
S-05	Hoch	Festes Standardpasswort für admin@example.com im Quellcode (DbSeeder.cs) und im README. Compose setzte kein eigenes Passwort, der Stack lief also mit diesem bekannten Zugang.	Kein Standardpasswort mehr: Ohne Seed:AdminPassword erzeugt das Backend ein Zufallspasswort und schreibt es einmalig ins Log. Das gilt nur für eine leere Datenbank – in der bestehenden funktioniert das alte Passwort weiterhin (geprüft).	Code behoben; Altpasswort akzeptiert (O-02)
S-06	Hoch	PostgreSQL (5432) und Backend (5000, ohne TLS) waren auf allen Netzwerkschnittstellen des Hosts veröffentlicht und damit am nginx vorbei erreichbar.	Beide Ports nur noch an 127.0.0.1 gebunden. Aus dem Netzwerk ist ausschliesslich nginx (80/443) erreichbar.	Behoben
S-07	Hoch	Kein Schutz gegen Passwort-Raten: Anmeldeversuche waren unbegrenzt möglich.	nginx begrenzt /auth/ und /api/auth/callback/credentials auf 10 Anfragen pro Minute und Client-IP (Antwort 429). Im Test wurden 21 von 25 schnellen Versuchen abgewiesen.	Behoben
S-08	Hoch	NuGet-Pakete mit bekannten Lücken (transitiv): Microsoft.Extensions.Caching.Memory, System.Text.Json (hoch), System.IdentityModel.Tokens.Jwt, Microsoft.IdentityModel.JsonWebTokens (mittel).	Pakete aktualisiert, danach Migration auf .NET 10. Die Prüfung meldet für beide Projekte keine anfälligen Pakete.	Behoben
S-09	Mittel	Open Redirect: Das Login-Formular leitete nach der Anmeldung ungeprüft auf den Parameter callbackUrl weiter, also auch auf fremde Seiten (Phishing über einen echten Login-Link).	Nur noch Pfade innerhalb der eigenen App werden akzeptiert, alles andere führt auf die Startseite.	Behoben
S-10	Mittel	Audit-Felder fälschbar: CreatedBy/ChangedBy wurden ersatzweise aus dem frei setzbaren Header X-User übernommen.	Der Header wird nicht mehr gelesen; massgebend ist allein der Name aus dem Token. Durch einen Test abgedeckt.	Behoben
S-11	Mittel	Benutzer-Enumeration: Der Login brauchte für unbekannte E-Mail-Adressen messbar länger (zusätzliche Hash-Berechnung), und die Registrierung antwortete für bestehende und neue Adressen unterschiedlich.	Fester, einmalig berechneter Vergleichs-Hash für unbekannte Benutzer; identische Antwort der Registrierung in beiden Fällen.	Behoben
S-12	Mittel	Backend- und Frontend-Container liefen als root.	USER in beiden Dockerfiles gesetzt (Backend: app, Frontend: node).	Behoben

S-13	Mittel	Frontend-Build ohne Lockfile: <code>npm install</code> holte bei jedem Build die jeweils neusten passenden Versionen, ohne Integritätsprüfung.	<code>package-lock.json</code> versioniert, Dockerfile verwendet <code>npm ci</code> .	Behoben
S-14	Niedrig	Unvollständige Sicherheitsheader; nginx gab seine Version preis.	Ergänzt: <code>Referrer-Policy</code> , <code>Permissions-Policy</code> , <code>Content-Security-Policy</code> (<code>frame-ancestors</code> , <code>base-uri</code> , <code>form-action</code> , <code>object-src</code>); bestehende Header auch auf Fehlerantworten; <code>server_tokens off</code> .	Behoben
S-15	Niedrig	Skripte <code>create_user.sh</code> und <code>reset_password.sh</code> : Eingaben wurden direkt in SQL und in Python-Quelltext eingesetzt, das Passwort am Ende im Klartext ausgegeben, Mindestlänge 6 statt 8.	Werte gehen als <code>psql</code> -Variablen bzw. über die Umgebung; keine Passwortausgabe; Mindestlänge 8. Mit einem Passwort und Namen voller Sonderzeichen getestet.	Behoben
S-16	Niedrig	PLZ-Suche: Ein Hochkomma in der Eingabe konnte den OData-Filter verändern (ohne Rechteausweitung).	Hochkommas werden verdoppelt.	Behoben
S-17	Niedrig	Keine Längenbegrenzung für E-Mail und Passwort bei Login und Registrierung.	<code>MaxLength</code> 256 bzw. 128 auf den DTOs.	Behoben
S-18	Niedrig	Ein echt wirkendes Passwort stand als «Standard» im Kommentar von <code>migration/import_cities.py</code> .	Aus der Datei entfernt.	Behoben (History: O-01, akzeptiert)

4. Akzeptierte Risiken und offene Punkte

O-01 und O-02 sind für die Demo-Anwendung akzeptiert. Bei einem Einsatz mit echten Daten sind sie vor der Inbetriebnahme zu erledigen. O-03 bis O-07 sind offene Restrisiken.

Nr.	Punkt	Stand und Empfehlung
O-01	Geheimnisse in der Git-History auf GitHub: <code>nginx/ssl/key.pem</code> , <code>frontend/.env.local</code> , Postman-Vault-Key, die alten Werte aus <code>docker-compose.yml</code> und das Passwort aus S-18.	Akzeptiert (Demo-Anwendung). JWT-Schlüssel, NextAuth-Secret und Datenbankpasswort sind dennoch bereits ersetzt. Bei einem Einsatz mit echten Daten: TLS-Zertifikat neu erzeugen und Postman-Vault-Key ersetzen. Unabhängig von der Demo-Anwendung gilt: Wird das Passwort aus S-18 an anderer Stelle verwendet, ist es dort zu ändern.
O-02	Das frühere Standardpasswort von <code>admin@example.com</code> gilt in der bestehenden Datenbank noch.	Akzeptiert (Demo-Anwendung). Bei einem Einsatz mit echten Daten <code>./scripts/reset_password.sh</code> ausführen.
O-03	OData-Listen ohne Seitengröße: <code>GET /odata/Cities</code> liefert alle rund 4000 Zeilen in einer Antwort; <code>MaxTop</code> begrenzt nur ausdrückliche <code>\$top</code> -Angaben.	<code>PageSize</code> setzen. Das verlangt Blättern im Frontend, das heute die ganze Liste in einer Antwort erwartet – deshalb nicht in dieser Runde.
O-04	Das Backend-Token ist für JavaScript im Browser lesbar (<code>session.accessToken</code>). Eine XSS-Lücke würde es preisgeben. Im Code wurde keine solche Lücke gefunden.	API-Aufrufe über Next.js-Routen leiten, damit das Token den Server nicht verlässt, und eine strikte <code>script-src</code> -CSP mit Nonces einführen.

O-05	Keine Rollen: Jeder angemeldete Benutzer darf alles, auch Einstellungen ändern und Benutzer anlegen.	Rolle «Admin» im Token einführen und Registrierung sowie Einstellungen darauf beschränken.
O-06	Keine Kontosperrung nach Fehlversuchen und kein Widerruf ausgestellter Tokens (8 Stunden gültig).	Sperrzähler pro Benutzer; kürzere Token-Laufzeit oder Sperrliste.
O-07	npm audit meldet noch 7 Einträge (hoch) in Entwicklungswerkzeugen, alle über braces (Tailwind CSS 3, ESLint). Sie laufen nur beim Build mit eigenen Dateien, nicht im ausgelieferten Frontend.	Mit einer Migration auf Tailwind CSS 4 beheben.

5. Überprüfung

Prüfung	Ergebnis
dotnet test	51 von 51 bestanden
dotnet list package --vulnerable --include-transitive	keine anfälligen Pakete
npm audit --omit=dev	0 Lücken (mit Entwicklungswerkzeugen: 7, siehe O-07)
Frontend-Build mit Next.js 15 (inkl. Typprüfung und Lint) im Docker-Image	erfolgreich
POST /auth/register ohne Token über nginx	401
Startseite ohne Session, auch mit dem Header x-middleware-subrequest	307 (Umleitung zum Login)
Anmeldung über NextAuth, Startseite und Städteliste mit Session	200, Daten vorhanden
25 schnelle Anmeldeversuche über nginx	4 × 401, 21 × 429
Antwort-Header auf /login	alle neuen Header vorhanden, Server: nginx ohne Version
Veröffentlichte Ports	5000 und 5432 nur an 127.0.0.1; 80/443 offen
Benutzer in den Containern	Backend UID 1654 (app), Frontend node
Anmeldung mit dem alten Standardpasswort an der bestehenden Datenbank	weiterhin möglich (O-02, akzeptiert)

Nicht im Browser geprüft: das Verhalten des Login-Formulars bei einer manipulierten `callbackUrl` (S-09) und die Bedienung der Formulare nach dem Upgrade auf React 19. Beides ist nur durch den erfolgreichen Build und die Aufrufe per `curl` abgedeckt.

6. Auswirkungen auf den Betrieb

- Der Stack braucht vor dem Start die Datei `.env`; auf diesem Rechner ist sie mit neuen Werten angelegt.
- Bestehende Sitzungen sind durch den neuen JWT-Schlüssel und das neue NextAuth-Secret ungültig; einmal neu anmelden.
- Lokales `dotnet run` braucht `Jwt__key` und `ConnectionStrings__DefaultConnection` aus der Umgebung (siehe README).

- Neue Benutzer entstehen über `./scripts/create_user.sh` oder durch einen angemeldeten Benutzer über `/auth/register`.